

MoonBit 开源生态项目贡献赛

项目申报书

MoonBit Depsight — 依赖健康诊断器

申报人	肖若愚
团队名称	鱼仔爱开发
GitHub 仓库	https://github.com/Tino-hue/moonmark
GitLink 仓库	https://www.gitlink.org.cn/LittleFish/moonbit-depsight/tree/main
主要实现语言	MoonBit
目标平台	mooncakes.io
项目类型	生态工具（CLI + 可视化报告）

一、项目名称

MoonBit Depsight — 依赖健康诊断器

二、项目简介

MoonBit Depsight 是一个面向 MoonBit 生态的依赖健康诊断 CLI 工具。它读取项目的 `moon.mod.json`，递归构建传递依赖图，从版本新鲜度、许可证合规、废弃 API 密度、体积合理性、维护活跃度五个维度对每个依赖包进行量化评分，并生成终端彩色报告、HTML 可视化报告和 JSON 结构化输出。

工具以纯 MoonBit 实现，编译为 WASM/JS 通过 Node.js 运行，计划发布至 mooncakes.io 供所有 MoonBit 开发者使用。

三、项目方向与适用场景

项目方向

生态工具 — 依赖审计、健康评分、风险可视化 CLI 工具。

适用场景

- 日常开发：开发者在添加新依赖前后运行 `depsight audit`，快速了解引入该依赖对项目健康度的影响。
- CI/CD 集成：在 GitHub Actions / GitLab CI 中配置 `depsight audit --fail-on-score 80 --fail-on-critical`，当依赖健康分低于阈值或存在 Critical 级别问题时中断流水线。
- 项目维护：维护者定期运行 `depsight report --html`，生成 HTML 可视化报告，直观审视依赖树的全貌与风险分布。
- 团队协作：通过 JSON 输出 (`depsight audit --json`) 接入内部仪表盘，持续追踪项目依赖健康趋势。

四、拟实现的核心功能

4.1 依赖树解析

- 读取本地 `moon.mod.json`，解析版本约束语法 (`^`、`~`、`~>`、`>=`、`>` 等)

- 递归拉取 mooncakes.io 上各包的元数据，构建完整的传递依赖图（DAG）
- 检测循环依赖并报告环路径

4.2 体积归因分析

- 统计每个传递依赖的自身大小与传递大小
- 按“体积罪魁祸首”降序排列，定位体积膨胀的根源包

4.3 废弃 API 检测

- 扫描依赖包源码或接口声明，提取 @deprecated 标记的公开 API
- 跨包传递检测：追踪依赖链中废弃 API 的间接暴露风险
- 生成诊断：Package B@0.2.0 -> Package A@0.1.0::old_function (deprecated since 0.1.5)

4.4 许可证合规

- 从 LICENSE 文件或 moon.mod.json 中自动识别 SPDX 协议标识符
- 支持识别 MIT、Apache-2.0、BSD-2/3-Clause、GPL-3.0、AGPL-3.0、LGPL-3.0、MPL-2.0、ISC、SSPL-1.0、Unlicense、CC0-1.0 等常见协议
- 标记 GPL、AGPL、SSPL 等强 copyleft 高风险协议

4.5 健康评分

五维加权评分模型（0-100 分）：

评分维度	权重	说明
版本新鲜度	25%	当前版本与最新版本的距离
协议合规性	20%	是否包含高风险协议
废弃 API 密度	25%	废弃 API 占总公开 API 的比例
体积合理性	20%	传递依赖体积是否在合理范围
维护活跃度	10%	预留维度，待接入 Git 提交活跃度

为根项目计算整体健康分（所有直接依赖的加权平均）。

4.6 报告输出

输出格式	命令	说明
终端报告	depsight audit	类似 npm audit，按 Critical/Warning/Info 分组，彩色表格 + 汇总
HTML 报告	depsight report --html	交互式依赖树 + 概览仪表盘 + 详细诊断列表
JSON 输出	depsight audit --json	供 CI/CD 流水线消费的结构化数据

输出格式	命令	说明
依赖树	depsight tree	树形缩进展示依赖层级，支持 --depth 参数

4.7 CI/CD 支持

- --fail-on-score <n>: 健康分低于阈值时返回非零退出码
- --fail-on-critical: 发现 Critical 级别诊断时返回非零退出码
- --offline: 仅使用本地缓存，不请求网络
- --cache-dir: 指定本地缓存路径

五、项目性质声明

本项目为 原创项目。

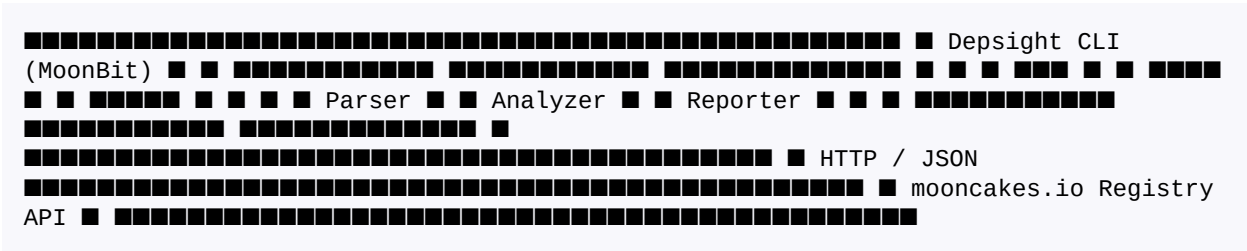
MoonBit Depsight 的核心设计（**多维健康评分模型**、**跨包废弃 API 传递分析**、**语义化体积归因**）均为独立构思与实现，不基于任何已有开源项目进行移植或参考。

开发过程中参考了以下工具的
输出格式和用户体验设计思路（仅参考交互形式，核心算法与实现完全独立）：

参考工具	参考内容	来源链接	许可证
npm audit	终端审计报告的分组格式与颜色方案	https://github.com/npm/cli	Artistic-2.0
cargo audit	CI 集成退出码策略	https://github.com/rustsec/rustsec	MIT / Apache-2.0
Snyk CLI	HTML 报告的仪表盘布局思路	https://github.com/snyk/snyk	Apache-2.0

六、技术方案

6.1 架构设计



6.2 核心模块

模块	实现语言	职责
moon.mod.json 解析器	MoonBit	读取本地依赖声明，处理版本约束语法
元数据获取器	MoonBit	并发拉取 mooncakes.io 上各包的 moon.mod.json 与接口声明
依赖图构建器	MoonBit	构建有向无环图（DAG），检测循环依赖
废弃 API 扫描器	MoonBit	基于接口文件或源码 AST，定位 @deprecated 标记
许可证识别器	MoonBit	解析 LICENSE 文件头，匹配 SPDX 标准协议标识符
报告渲染器	MoonBit→JS/WASM	生成终端表格（TTY）与 HTML 可视化报告

6.3 关键技术选型

- MoonBit 标准库：@moonbitlang/core 提供 JSON 解析、字符串处理、集合操作，无需引入额外依赖。
- 编译目标：CLI 主体编译为 WASM/JS，通过 Node.js 运行；核心算法模块编译为 WASM-GC 以追求性能。
- HTTP 客户端：使用 MoonBit JS FFI 调用 fetch 获取 mooncakes.io API 数据。
- 输出格式：支持终端富文本、静态 HTML 报告、JSON（供 CI/CD 消费）。

七、创新点

1. 跨包废弃 API 传递分析

不仅检测直接依赖的废弃接口，更追踪“你的依赖所依赖的包”中是否存在已废弃 API 的跨层调用，这是现有任何 MoonBit 工具都不具备的能力。

2. 语义化体积归因

不是简单统计源码体积，而是结合 MoonBit 的编译特性，估算各传递依赖对最终 .wasm 产物的体积贡献（基于符号引用分析）。

3. 依赖健康评分模型

设计一套可量化的评分算法（0-100 分），综合考量版本新鲜度、协议兼容性、废弃 API 密度、维护活跃度，让“健康度”一目了然。

4. 原生 MoonBit 实现

核心解析器、图算法、分析引擎全部使用 MoonBit 编写，充分展示语言在系统工具开发上的表达能力，并反哺标准库的实战验证。

附录

项目	内容
比赛名称	MoonBit × CCF 开源生态项目贡献赛
技术栈	MoonBit / WASM / JavaScript FFI / HTML/CSS
开源协议	Apache-2.0
依赖声明	除 @moonbitlang/core 外，不引入第三方 MoonBit 依赖；构建时工具链使用 Node.js 18+